



**KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN MẠNG VÀ CÁC HỆ THỐNG THÔNG TIN**

CHƯƠNG 9

QUẢN LÝ THIẾT BỊ

Nội dung chính - Chương 9



Mục lục

- 9.1. Nguyên lý hoạt động
- 9.2. Chiến lược quản lý thiết bị
- 9.3. Trình điều khiển thiết bị

Từ khóa

I/O	Controller
Interrupt	DMA
Buffer	Spooling
Scheduling	Driver
Device table	Plug and Play
Block device	Character device
HDD • Băng từ • SCSI • RAID • SSD	



Hiểu luồng vào/ra

Mô tả cách ứng dụng gửi yêu cầu đến hệ điều hành, bộ điều khiển và thiết bị.

Quản lý hợp lý

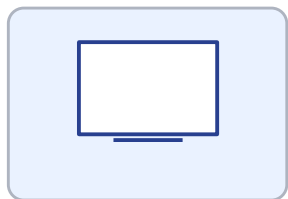
Giải thích cách cấp phát, xếp hàng và lập lịch để nhiều tiến trình dùng thiết bị hiệu quả.

Hiểu trình điều khiển

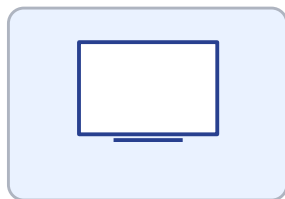
Mô tả vai trò của driver và cách driver giao tiếp với thiết bị.

Câu hỏi định hướng: Làm thế nào để nhiều chương trình dùng các thiết bị khác nhau mà vẫn nhanh, đúng và an toàn?

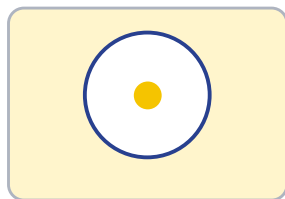
Một máy tính có rất nhiều loại thiết bị



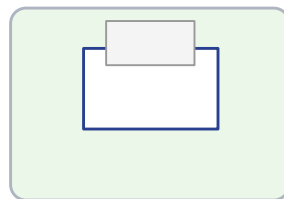
Bàn phím



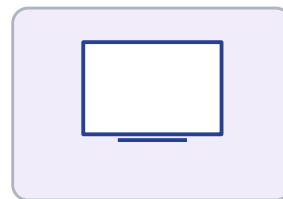
Màn hình



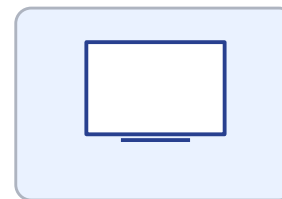
Ổ đĩa



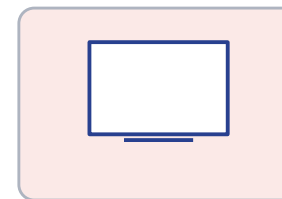
Máy in



Card mạng



USB



Cảm biến

Mỗi thiết bị có tốc độ, cách truyền dữ liệu và tín hiệu điều khiển khác nhau. Hệ điều hành phải tạo ra một cách sử dụng thống nhất để ứng dụng không cần biết mọi chi tiết phần cứng.



9.1. Nguyên lý hoạt động

Thiết bị trao đổi dữ liệu với CPU và bộ nhớ thông qua bộ điều khiển, tín hiệu ngắt và các vùng đệm.



Vì sao hệ điều hành phải quản lý thiết bị?

Khác nhau

Mỗi thiết bị dùng lệnh và tốc độ khác nhau.

Chậm hơn CPU

CPU có thể phải chờ nếu trao đổi không hợp lý.

Dùng chung

Nhiều chương trình có thể yêu cầu cùng một thiết bị.

Có lỗi

Thiết bị có thể hết giấy, mất kết nối hoặc truyền sai.

Hệ điều hành che giấu khác biệt phần cứng và điều phối các yêu cầu vào/ra.



Các thiết bị khác nhau ở những điểm nào?

Đặc điểm	Ví dụ	Ảnh hưởng đến quản lý
Đơn vị dữ liệu	Bàn phím theo ký tự; đĩa theo khối	Cần giao diện đọc/ghi khác nhau
Tốc độ	Bàn phím chậm; SSD nhanh	Cần bộ đệm và lập lịch phù hợp
Cách truy cập	Tuần tự hoặc ngẫu nhiên	Ảnh hưởng cách xếp yêu cầu
Khả năng chia sẻ	Máy in thường dùng lần lượt	Cần hàng đợi và quyền sử dụng
Loại lỗi	Mất kết nối, hết giấy, bad block	Cần phát hiện và khôi phục

Các lớp trong hệ thống vào/ra



Ứng dụng

read(), write(), print...

Giao diện hệ điều hành

Kiểm tra quyền, tham số và trạng thái

Trình điều khiển thiết bị

Chuyển yêu cầu chung thành lệnh phần cứng

Bộ điều khiển thiết bị

Thanh ghi, bộ đệm và mạch điều khiển

Thiết bị vật lý

Đĩa, máy in, bàn phím, card mạng...



Một yêu cầu đọc dữ liệu đi qua những bước nào?



Ví dụ: Trình phát nhạc yêu cầu đọc một phần tệp. Hệ điều hành không tự đọc trực tiếp ổ đĩa mà giao việc cho driver và bộ điều khiển.

Ứng dụng nhìn thấy thao tác đơn giản; phần phức tạp được xử lý ở các lớp bên dưới.

Bộ điều khiển thiết bị là gì?



Thiết bị vật lý

- Phần thực hiện công việc thật: quay đĩa, in giấy, truyền tín hiệu...
- Có đặc điểm cơ khí hoặc điện tử riêng.

Bộ điều khiển

- Mạch điện tử đứng giữa CPU và thiết bị.
- Nhận lệnh, lưu trạng thái, truyền dữ liệu và báo hoàn thành.

CPU thường làm việc với bộ điều khiển thay vì điều khiển trực tiếp mọi chi tiết của thiết bị.

CPU giao tiếp với bộ điều khiển qua các thanh ghi



Thanh ghi dữ liệu

Chứa byte hoặc từ dữ liệu đang được gửi hay nhận.

Thanh ghi trạng thái

Cho biết thiết bị đang bận, đã xong hoặc gặp lỗi.

Thanh ghi điều khiển

Nhận lệnh như đọc, ghi, bắt đầu, dừng hoặc đặt chế độ.



Cách 1: Thăm dò liên tục (Polling)



Ưu điểm

Cách làm đơn giản, dễ kiểm soát.

Hạn chế

CPU mất thời gian hỏi đi hỏi lại dù thiết bị chưa xong.

Phù hợp

Thiết bị rất nhanh hoặc hệ thống đơn giản, ít công việc khác.

Cách 2: Thiết bị báo bằng ngắt

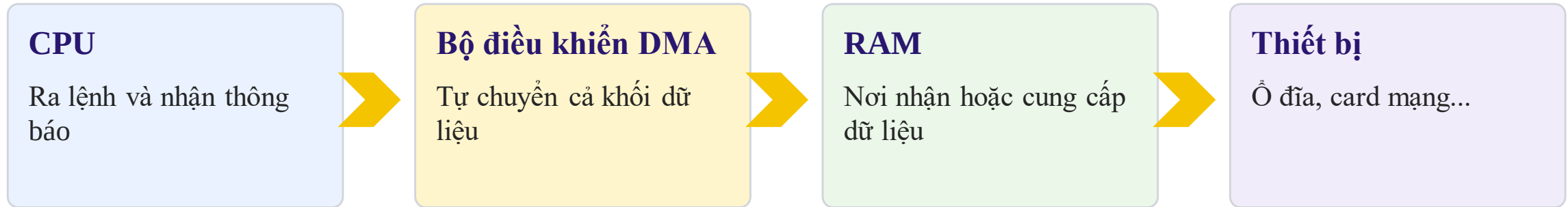


Khi có ngắt, CPU tạm dừng công việc hiện tại, lưu trạng thái cần thiết, chạy chương trình xử lý ngắt rồi quay lại công việc cũ.

Ngắt giúp CPU không phải đứng chờ hoặc kiểm tra liên tục.



Cách 3: DMA truyền dữ liệu trực tiếp với bộ nhớ



CPU vẫn tham gia lúc bắt đầu và khi kết thúc, nhưng phần truyền dữ liệu chính được DMA đảm nhiệm.

DMA phù hợp khi cần truyền nhiều dữ liệu, vì CPU không phải xử lý từng byte.

So sánh ba cách trao đổi với thiết bị



Cách làm	CPU phải chờ?	Ai chuyển dữ liệu?	Phù hợp
Polling	Thường có	CPU	Thiết bị đơn giản hoặc rất nhanh
Ngắt	Không phải chờ liên tục	CPU sau mỗi lần ngắt	Dữ liệu nhỏ, sự kiện không đều
DMA	Chỉ khởi tạo và nhận kết quả	Bộ điều khiển DMA	Khối dữ liệu lớn, tốc độ cao

Không có cách nào luôn tốt nhất; hệ điều hành chọn theo loại thiết bị và lượng dữ liệu.



Programmed I/O: CPU chuyển từng phần dữ liệu

Cách hoạt động

- CPU đọc/ghi trực tiếp thanh ghi dữ liệu của bộ điều khiển.
- Mỗi byte hoặc từ dữ liệu cần CPU tham gia.
- Có thể kết hợp polling hoặc ngắt.

Đánh giá

- Đơn giản và dễ cài đặt.
- Tốn nhiều chu kỳ CPU khi dữ liệu lớn.
- Thường dùng cho thiết bị chậm hoặc lượng dữ liệu nhỏ.

DMA giảm gánh nặng cho CPU khi cần truyền cả khối dữ liệu lớn.



Hai cách đặt địa chỉ cho thanh ghi thiết bị

Memory-mapped I/O

- Thanh ghi thiết bị nằm trong cùng không gian địa chỉ với bộ nhớ.
- CPU dùng lệnh đọc/ghi bộ nhớ thông thường.
- Lập trình thuận tiện, nhưng phải kiểm soát cache và quyền truy cập.

Cổng vào/ra riêng

- Thiết bị có không gian địa chỉ riêng.
- CPU dùng lệnh vào/ra chuyên dụng.
- Tách rõ bộ nhớ và thiết bị, nhưng cần tập lệnh hỗ trợ.

Cả hai đều giúp CPU tìm đúng thanh ghi của bộ điều khiển.



Đồng bộ và bất đồng bộ trong vào/ra

Vào/ra đồng bộ

- Lời gọi chỉ trả về khi thao tác đã hoàn thành.
- Dễ hiểu, dễ lập trình.
- Tiến trình có thể phải chờ.

Vào/ra bất đồng bộ

- Lời gọi trả về sớm; kết quả được báo sau.
- Tiến trình có thể tiếp tục làm việc khác.
- Cần cơ chế thông báo và quản lý kết quả.

“Đồng bộ” ở đây nói về thời điểm chương trình nhận kết quả, không phải tốc độ của thiết bị.



Blocking và non-blocking khác nhau thế nào?

Blocking

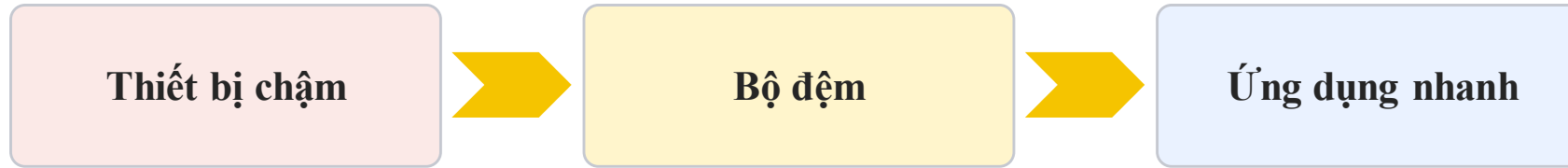
- Tiến trình bị chuyển sang trạng thái chờ khi chưa có dữ liệu.
- CPU chạy tiến trình khác.
- Phù hợp khi chương trình cần kết quả mới làm tiếp.

Non-blocking

- Lời gọi trả về ngay, có thể báo “chưa có dữ liệu”.
- Chương trình tự thử lại hoặc làm công việc khác.
- Phù hợp ứng dụng tương tác và máy chủ.

Blocking không có nghĩa là CPU bị đứng; chỉ tiến trình yêu cầu bị chờ.

Bộ đệm giúp cân bằng tốc độ



Giảm chờ

Ứng dụng và thiết bị không cần hoạt động đúng cùng tốc độ.

Gom dữ liệu

Nhiều phần nhỏ có thể được gom thành một lần truyền lớn.

Giữ tạm

Dữ liệu được lưu tạm khi bên nhận chưa sẵn sàng.



Một bộ đệm, hai bộ đệm và bộ đệm vòng

Một bộ đệm

Thiết bị và chương trình dùng luân phiên một vùng nhớ. Dễ làm nhưng có lúc phải chờ nhau.

Hai bộ đệm

Trong khi một vùng đang được xử lý, vùng còn lại có thể nhận dữ liệu mới.

Bộ đệm vòng

Nhiều vùng nối thành vòng, phù hợp luồng dữ liệu liên tục như âm thanh và mạng.

Nhiều bộ đệm tăng tính song song nhưng cần thêm bộ nhớ và quản lý chỉ số đọc/ghi.



Cache và buffer không hoàn toàn giống nhau

Buffer

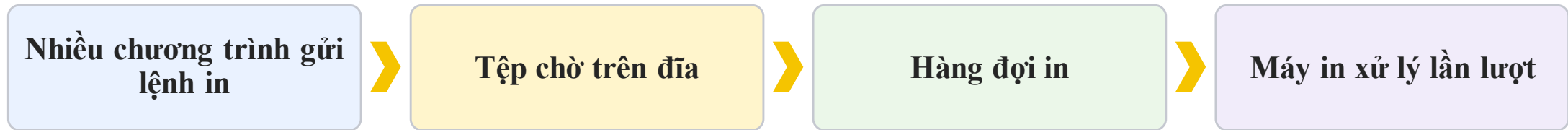
- Giữ dữ liệu tạm để cân bằng tốc độ.
- Dữ liệu thường đang trên đường từ nguồn đến đích.
- Ví dụ: vùng nhận gói tin mạng.

Cache

- Giữ lại bản sao dữ liệu đã dùng để truy cập nhanh lần sau.
- Có thể bỏ và đọc lại từ thiết bị gốc.
- Ví dụ: cache khối đĩa.

Một vùng nhớ có thể vừa đóng vai trò buffer vừa đóng vai trò cache trong một số hệ thống.

Spooling: xếp công việc vào vùng chờ



Máy in chỉ thực hiện một công việc tại một thời điểm. Spooling cho phép các chương trình gửi dữ liệu nhanh rồi tiếp tục chạy, còn dịch vụ in xử lý hàng đợi phía sau.

Spooling biến một thiết bị khó chia sẻ thành dịch vụ dùng chung thông qua hàng đợi.



Ví dụ: ba sinh viên cùng gửi lệnh in

Thời điểm	Yêu cầu	Trạng thái hàng đợi
10:00:00	A gửi 12 trang	A
10:00:03	B gửi 2 trang	$A \rightarrow B$
10:00:06	C gửi 6 trang	$A \rightarrow B \rightarrow C$
10:01:10	A in xong	$B \rightarrow C$
10:01:25	B in xong	C

Ứng dụng không cần giữ máy in; hệ điều hành lưu công việc và phục vụ theo chính sách hàng đợi.



CPU xử lý một tín hiệu ngắt theo các bước



Nhanh

Phần xử lý đầu tiên cần ngắn để không chặn các ngắt khác.

Đúng nguồn

Hệ điều hành phải biết thiết bị nào phát ngắt và vì sao.

Phần việc sau

Công việc dài có thể chuyển sang luồng hoặc hàng đợi xử lý sau.

Trạng thái thiết bị cần được theo dõi



Ready

Thiết bị sẵn sàng nhận lệnh mới.

Busy

Thiết bị đang xử lý yêu cầu trước.

Done

Thao tác đã hoàn thành.

Error

Có lỗi và cần xem mã lỗi chi tiết.

Driver đọc các bit trạng thái để quyết định tiếp tục, chờ, thử lại hay báo lỗi cho ứng dụng.

Khi thiết bị gặp lỗi, hệ điều hành làm gì?



Lỗi tạm thời

Ví dụ nhiễu mạng hoặc thiết bị đang bận: có thể thử lại.

Lỗi cần người dùng

Ví dụ máy in hết giấy: yêu cầu bổ sung giấy.

Lỗi nghiêm trọng

Ví dụ thiết bị mất kết nối: hủy yêu cầu và báo lỗi.



9.2. Chiến lược quản lý thiết bị

Hệ điều hành cần biết thiết bị nào đang rồi, ai đang sử dụng, yêu cầu nào chờ trước và thứ tự phục vụ ra sao.

Mục tiêu của quản lý thiết bị



Hiệu quả

Giảm thời gian chờ và tận dụng thiết bị tốt.

Công bằng

Không để một tiến trình chờ quá lâu.

An toàn

Ngăn truy cập trái phép hoặc xung đột.

Dễ sử dụng

Cung cấp giao diện thống nhất cho ứng dụng.

Một chính sách tốt phải cân bằng tốc độ, công bằng và độ tin cậy.



Có thể phân loại thiết bị theo cách sử dụng

Thiết bị riêng

Một tiến trình dùng trong một khoảng thời gian. Ví dụ máy in đang in một công việc.

Thiết bị dùng chung

Nhiều tiến trình có thể truy cập xen kẽ. Ví dụ ổ đĩa và card mạng.

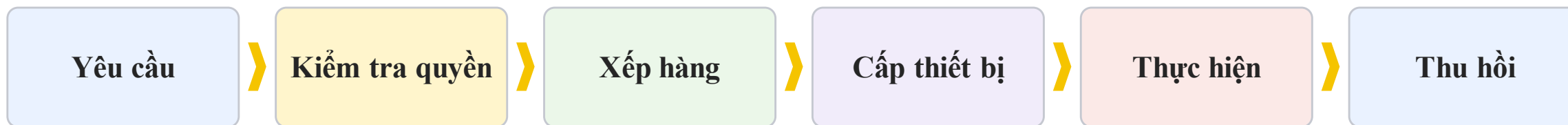
Thiết bị ảo

Hệ điều hành tạo cảm giác có nhiều thiết bị từ một thiết bị thật. Ví dụ máy in ảo, ổ đĩa ảo.

Loại	Cách dùng	Ví dụ	Điểm cần quản lý
Riêng	Một yêu cầu chiếm giữ trong một khoảng thời gian	Máy in vật lý	Cấp phát và thu hồi rõ ràng
Dùng chung	Nhiều yêu cầu được xen kẽ hoặc xếp hàng	Ổ đĩa, card mạng	Lập lịch và bảo vệ dữ liệu
Ảo hóa	Mỗi tiến trình thấy một thiết bị logic riêng	Terminal ảo, máy in ảo	Ánh xạ thiết bị ảo sang tài nguyên thật

Thiết bị ảo giúp chia sẻ linh hoạt và giảm phụ thuộc vào phần cứng cụ thể.

Vòng đời một yêu cầu thiết bị



Nếu thiết bị đang bận, yêu cầu được đưa vào hàng đợi. Khi thiết bị rỗi, hệ điều hành chọn yêu cầu tiếp theo theo chính sách đã đặt.

Thu hồi đúng lúc giúp yêu cầu khác có cơ hội sử dụng thiết bị.

Bảng thiết bị lưu những thông tin gì?

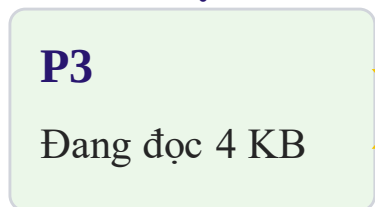


Trường thông tin	Ý nghĩa	Ví dụ
Tên / mã thiết bị	Giúp hệ điều hành xác định đúng thiết bị	disk0, printer1
Loại thiết bị	Chọn cách xử lý phù hợp	Khối, ký tự, mạng
Trạng thái	Rỗi, bận, lỗi hoặc ngoại tuyến	Busy
Driver phụ trách	Hàm xử lý thiết bị	nvme_driver
Hàng đợi yêu cầu	Các tiến trình đang chờ	P2 → P5 → P1
Thông số phần cứng	Địa chỉ thanh ghi, IRQ, kênh DMA	IRQ 11

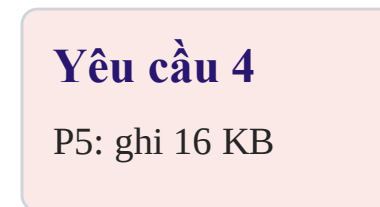
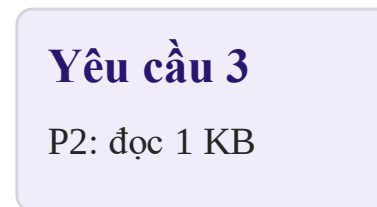
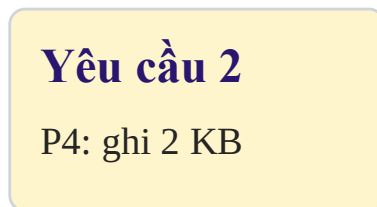
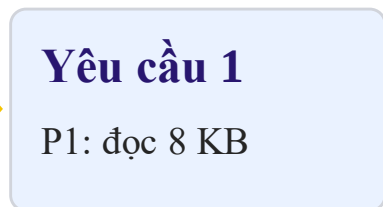
Hàng đợi yêu cầu giúp điều phối thiết bị



Thiết bị đang phục vụ

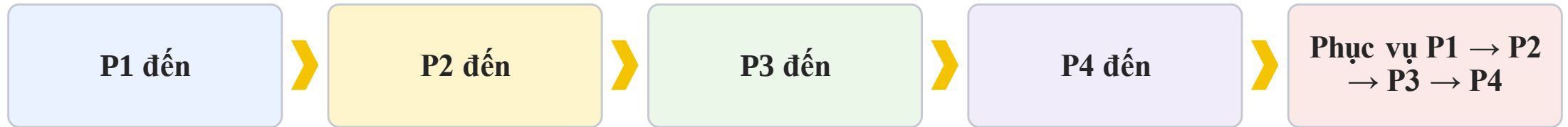


Các yêu cầu đang chờ



Chính sách lập lịch quyết định yêu cầu nào được lấy ra khỏi hàng đợi trước.

FCFS: đến trước phục vụ trước



Ưu điểm

Đơn giản, dễ hiểu và không thiên vị yêu cầu đến sau.

Hạn chế

Một yêu cầu chậm có thể làm các yêu cầu nhỏ chờ lâu.

Phù hợp

Thiết bị không cần tối ưu vị trí truy cập hoặc tải không cao.

Lập lịch theo mức ưu tiên



Yêu cầu	Mức ưu tiên	Thứ tự đến	Thứ tự phục vụ
P1 đọc log	2	1	3
P2 xử lý âm thanh	5	2	1
P3 sao lưu	1	3	4
P4 giao tiếp người dùng	4	4	2

Cần cơ chế tăng dần ưu tiên cho yêu cầu chờ lâu để tránh “chết đói”.

Lập lịch công bằng giữa nhiều tiến trình



Theo lượt

Mỗi tiến trình được phục vụ một phần rồi chuyển lượt.

Theo hạn ngạch

Giới hạn số yêu cầu hoặc lượng dữ liệu trong mỗi khoảng thời gian.

Theo thời gian chờ

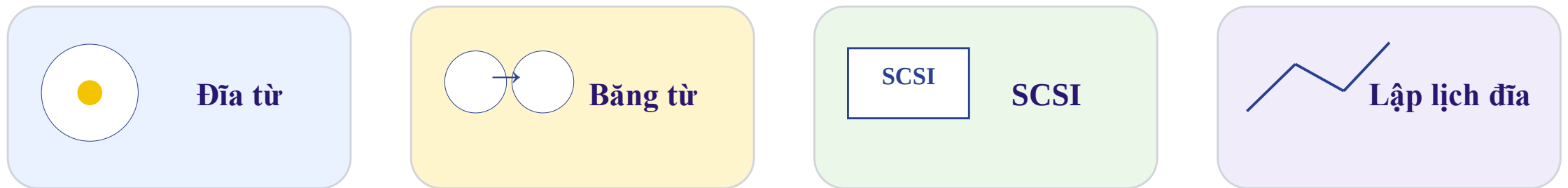
Yêu cầu chờ lâu được tăng cơ hội phục vụ.

Công bằng không nhất thiết nghĩa là mọi yêu cầu có thời gian bằng nhau; có thể cần ưu tiên cho tác vụ tương tác hoặc thời gian thực.



9.2.1. Thiết bị lưu trữ phụ

Đĩa từ, băng từ, SCSI và các yếu tố ảnh hưởng đến việc tổ chức, truy cập và lập lịch dữ liệu.

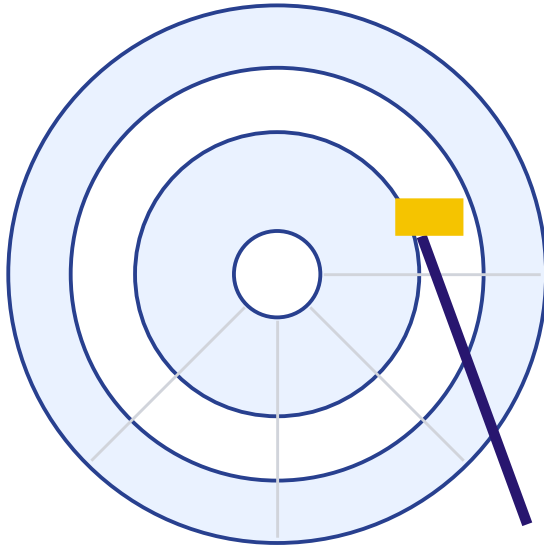


Hệ điều hành làm việc với các khối luận lý; bộ điều khiển và driver xử lý chi tiết phần cứng.

Đĩa từ lưu dữ liệu như thế nào?



Cấu tạo cơ bản



Phiên đĩa (platter)

Bề mặt phủ vật liệu từ, quay quanh trục với tốc độ ổn định.

Rãnh và sector

Mỗi mặt được chia thành các rãnh đồng tâm; mỗi rãnh gồm nhiều sector.

Đầu đọc/ghi

Di chuyển theo bán kính để đến đúng rãnh và đọc hoặc ghi dữ liệu.

Khối đĩa

Hệ điều hành thường trao đổi dữ liệu theo khối, không theo từng bit riêng lẻ.

Rãnh (track) • Cung (sector) • Đầu đọc/ghi

Vị trí vật lý của dữ liệu ảnh hưởng trực tiếp đến thời gian truy cập của HDD.



Từ vị trí vật lý đến địa chỉ khối luận lý

CHS

Cylinder – Head – Sector: mô tả trụ, mặt đĩa và sector. Cách biểu diễn này gắn với hình học của đĩa.

LBA

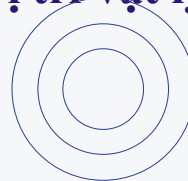
Logical Block Addressing: đánh số các khối liên tiếp 0, 1, 2, ... để hệ điều hành truy cập đơn giản hơn.

Không gian khối luận lý



Bộ điều khiển ánh xạ

Vị trí vật lý trên thiết bị



Ý nghĩa với hệ điều hành

- Đọc/ghi theo số khối.
- Không cần biết chi tiết hình học.
- Controller có thể thay thế bad block.
- Driver chuyển yêu cầu thành lệnh thiết bị.

LBA giúp tách giao diện của hệ điều hành khỏi cấu trúc vật lý cụ thể của ổ đĩa.



Thời gian truy cập của đĩa từ gồm những phần nào?

Tìm rãnh

Seek time

Đầu đọc di chuyển đến đúng rãnh.

Chờ quay

Rotational latency

Đợi sector cần đọc quay tới đầu đọc.

Truyền dữ liệu

Transfer time

Dữ liệu đi giữa đĩa và bộ điều khiển.

Chờ hàng đợi

Queue time

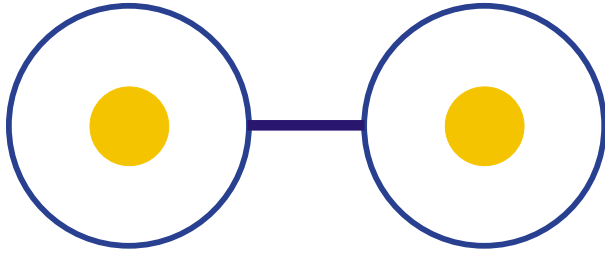
Yêu cầu chờ các thao tác trước hoàn thành.

$$T \text{ truy cập} \approx T \text{ tìm rãnh} + T \text{ chờ quay} + T \text{ truyền} + T \text{ hàng đợi}$$

Lập lịch đĩa chủ yếu giảm quãng đường tìm rãnh và thời gian chờ trong hàng đợi.



Băng từ: lưu trữ tuần tự, phù hợp sao lưu dài hạn



Dữ liệu được đọc theo thứ tự dọc băng

Ưu điểm

Dung lượng lớn, chi phí trên mỗi TB thấp, thích hợp lưu trữ ngoại tuyến và lưu trữ lâu dài.

Hạn chế

Truy cập ngẫu nhiên chậm; phải định vị hoặc tua đến vùng dữ liệu cần đọc.

Ứng dụng

Sao lưu định kỳ, lưu trữ hồ sơ, dữ liệu ít truy cập nhưng cần giữ nhiều năm.

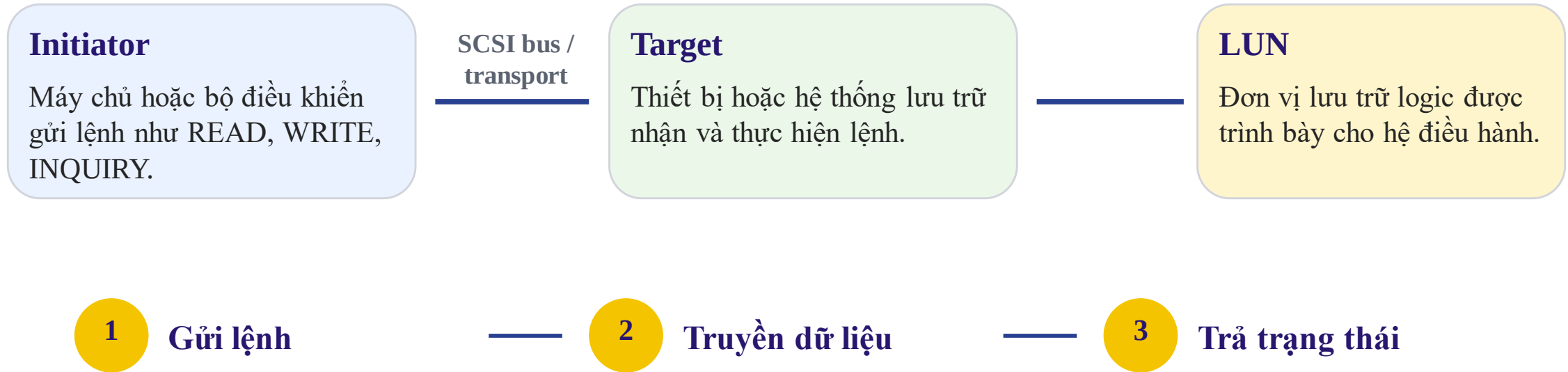
Quản lý

Hệ điều hành và phần mềm sao lưu tổ chức dữ liệu theo phiên, tệp và vị trí trên băng.

Băng từ ưu tiên dung lượng và chi phí; đĩa từ ưu tiên khả năng truy cập trực tiếp.



SCSI: mô hình lệnh và giao tiếp với thiết bị lưu trữ



SCSI có thể được vận chuyển qua nhiều công nghệ: SCSI song song, SAS hoặc iSCSI qua mạng IP.

SCSI không chỉ là một loại đầu cắm; cốt lõi là tập lệnh và mô hình initiator–target–LUN.



Lập lịch đĩa: chọn thứ tự phục vụ các yêu cầu

Hàng đợi yêu cầu

98	183	37	122
14	124	65	67

Bộ lập lịch đĩa
FCFS • SSTF • SCAN...

Thứ tự phục vụ
Giảm di chuyển đầu đọc

Hiệu năng

Giảm tổng quãng đường và thời gian chờ trung bình.

Công bằng

Không để yêu cầu ở xa phải chờ quá lâu.

Ổn định

Giữ thời gian đáp ứng dự đoán được khi tải cao.

Phù hợp thiết bị

HDD cần tối ưu vị trí; SSD ít phụ thuộc vị trí vật lý.

Các slide tiếp theo minh họa FCFS, SSTF, SCAN, C-SCAN, LOOK và C-LOOK.



Vì sao ổ đĩa cần thuật toán lập lịch riêng?

Ổ đĩa từ

Đầu đọc phải di chuyển đến đúng rãnh; khoảng cách ảnh hưởng thời gian.

Nhiều yêu cầu

Các yêu cầu có thể nằm ở nhiều vị trí khác nhau trên đĩa.

Mục tiêu

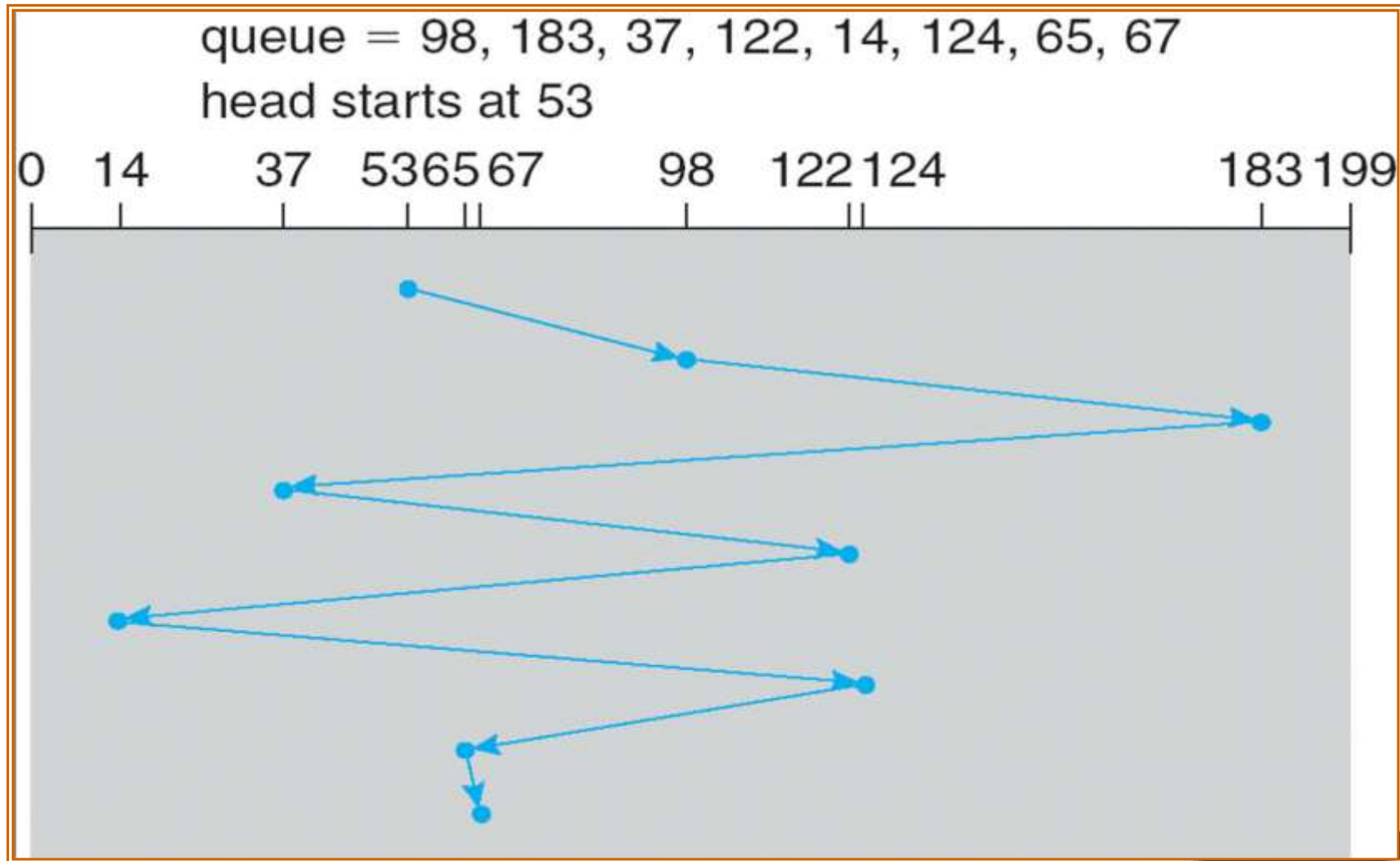
Giảm tổng quãng đường di chuyển và thời gian chờ trung bình.

Với SSD, thời gian truy cập ít phụ thuộc vị trí hơn, nhưng hàng đợi vẫn cần được quản lý.

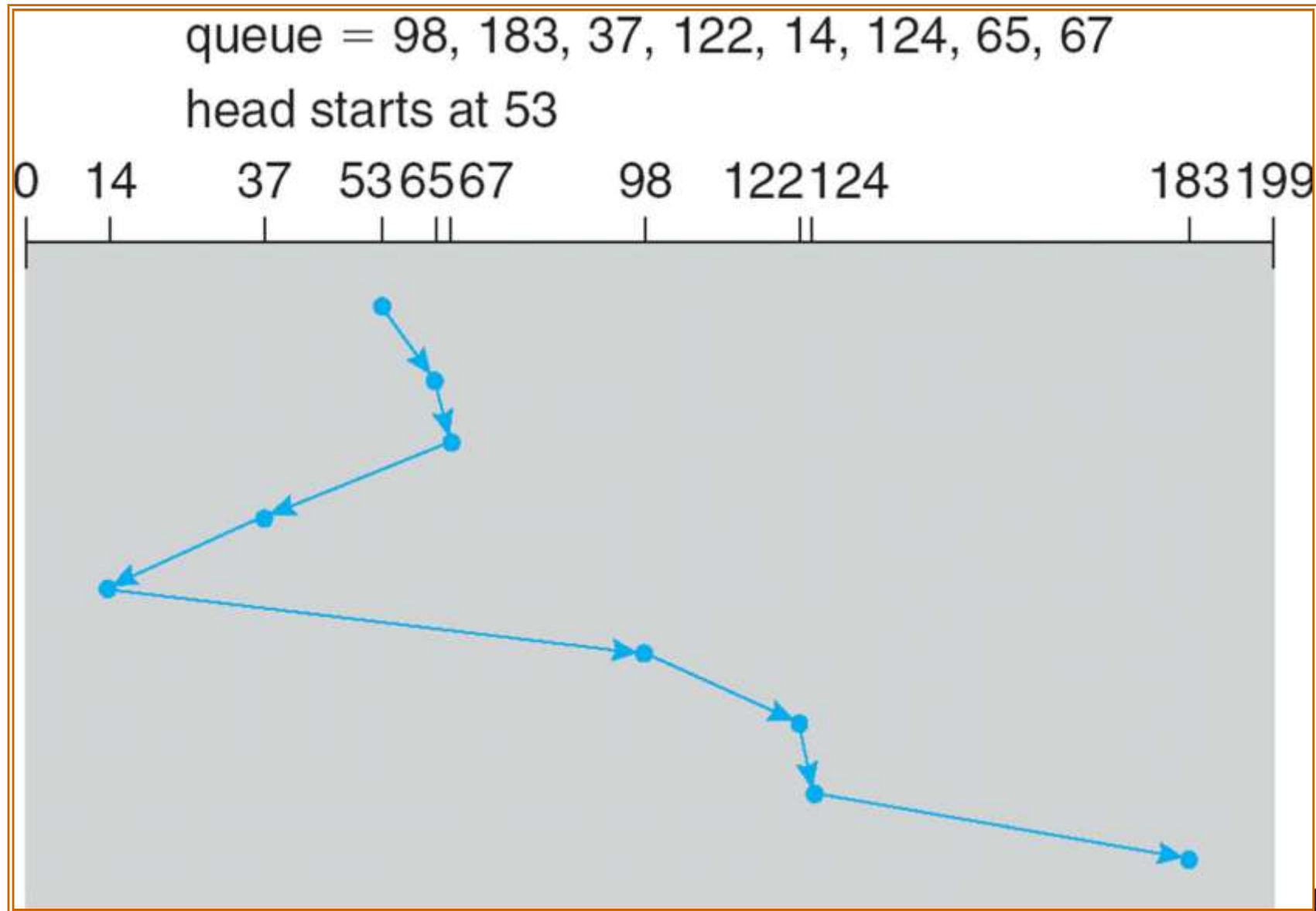


Ví dụ FCFS cho hàng đợi ổ đĩa

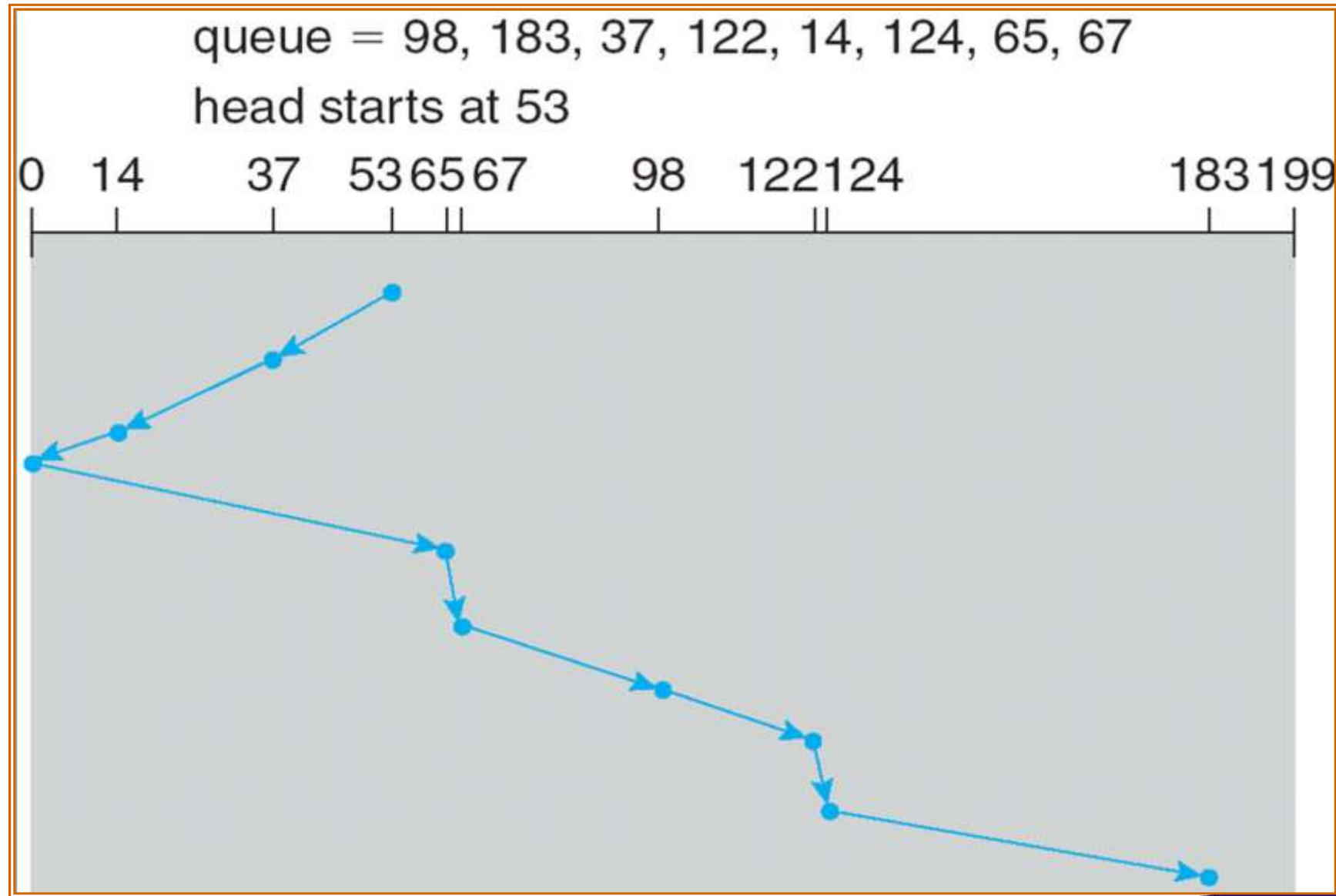
Đầu đọc bắt đầu tại rãnh 53. Hàng đợi: 98, 183, 37, 122, 14, 124, 65, 67



SSTF: chọn yêu cầu gần vị trí hiện tại nhất



SCAN: đầu đọc di chuyển như thang máy



SCAN: đầu đọc di chuyển như thang máy



Để hình dung

Giống thang máy đi lên rồi đi xuống.

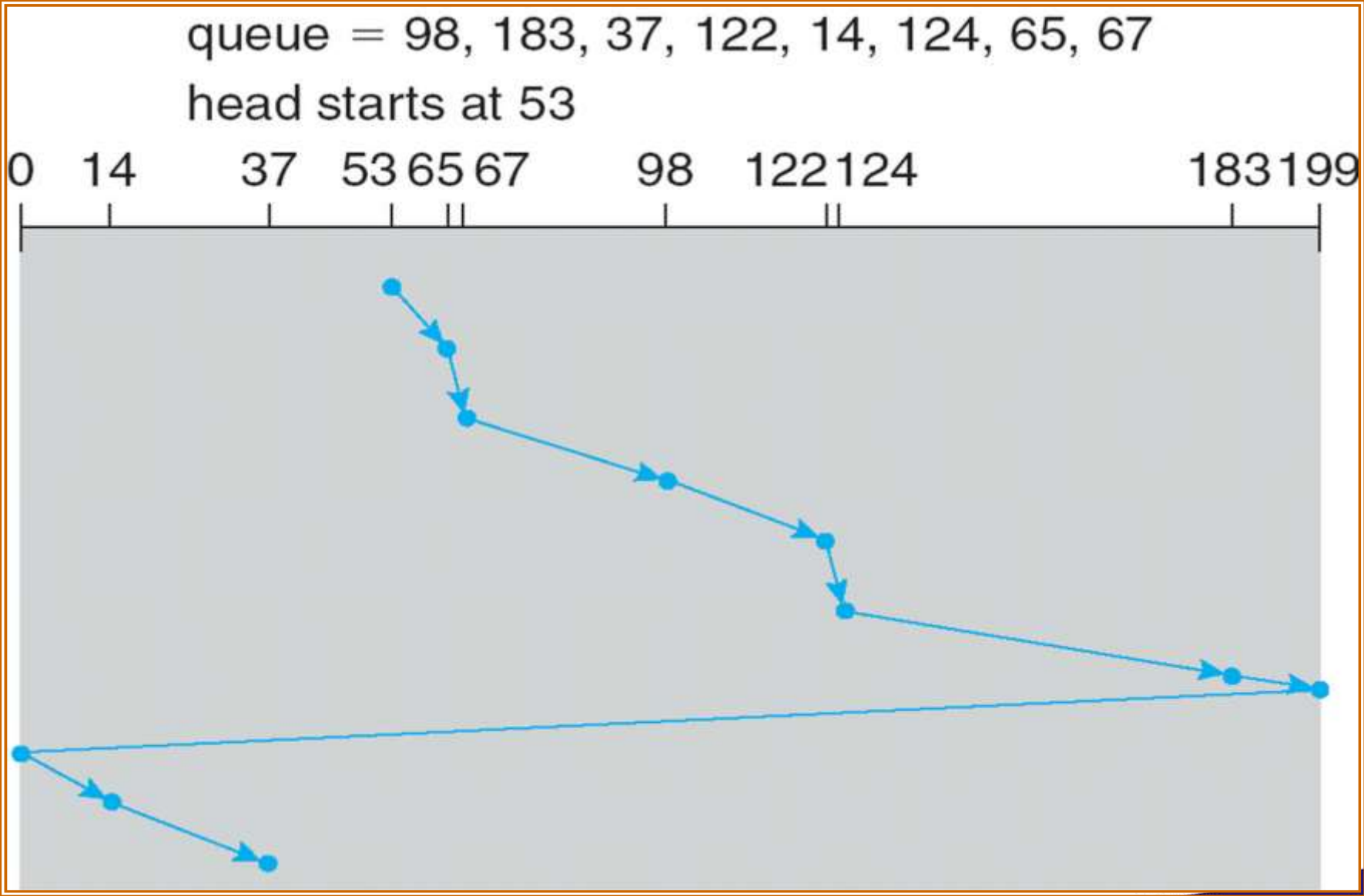
Ổn định

Giới hạn thời gian chờ tốt hơn SSTF trong tải cao.

Lưu ý

Cần biết hướng hiện tại và phạm vi cuối đĩa.

C-SCAN: chỉ phục vụ theo một hướng



C-SCAN: chỉ phục vụ theo một hướng



Khi quay về đầu, đầu đọc không phục vụ yêu cầu trên đường quay lại.

C-SCAN tạo thời gian chờ đồng đều hơn giữa các vị trí trên đĩa.



LOOK và C-LOOK không đi tới mép đĩa nếu không cần

LOOK

- Di chuyển theo một hướng đến yêu cầu xa nhất đang chờ.
- Sau đó đổi hướng.
- Không đi đến rãnh cuối nếu không có yêu cầu.

C-LOOK

- Chỉ phục vụ theo một hướng.
- Đến yêu cầu xa nhất rồi nhảy về yêu cầu thấp nhất.
- Không cần chạm mép vật lý của đĩa.

LOOK và C-LOOK giảm quãng đường thừa so với SCAN và C-SCAN.



So sánh nhanh các thuật toán lập lịch đĩa

Thuật toán	Ý tưởng	Ưu điểm	Hạn chế
FCFS	Theo thứ tự đến	Đơn giản, công bằng	Di chuyển nhiều
SSTF	Chọn vị trí gần nhất	Giảm quãng đường	Có thể làm yêu cầu xa chờ lâu
SCAN	Đi hai chiều như thang máy	Ổn định khi tải cao	Có thể đi đến mép không cần thiết
C-SCAN	Chỉ phục vụ một chiều	Thời gian chờ đồng đều	Có quãng quay về
LOOK / C-LOOK	Chỉ đi đến yêu cầu cuối	Giảm di chuyển thừa	Quản lý phức tạp hơn FCFS

RAID: kết hợp nhiều đĩa thành một hệ thống lưu trữ



RAID phân bố dữ liệu trên nhiều ổ đĩa để tăng tốc độ, tăng khả năng chịu lỗi hoặc kết hợp cả hai.

Striping

Chia dữ liệu thành dải và ghi lên nhiều đĩa song song.

1

2

3

Mirroring

Ghi bản sao giống nhau lên hai hoặc nhiều đĩa.

1

2

3

Parity

Lưu thông tin kiểm tra để khôi phục khi một số đĩa hỏng.

1

2

3

Mỗi mức RAID có cách đánh đổi khác nhau giữa dung lượng, hiệu năng, chi phí và chịu lỗi.



RAID 0 và RAID 1: tốc độ hoặc bản sao

RAID 0 – Striping

Đĩa 1

A1
A3
A5
A7

Đĩa 2

A2
A4
A6
A8

- Đọc/ghi song song
- Dùng toàn bộ dung lượng
- Không chịu lỗi
- Hỏng 1 đĩa có thể mất mảng

RAID 1 – Mirroring

Đĩa 1

A1
A2
A3
A4

Đĩa 2

A1
A2
A3
A4

- Có bản sao tức thời
- Đọc có thể nhanh hơn
- Chịu hỏng 1 đĩa trong cặp
- Dung lượng sử dụng khoảng 50%

RAID 0 ưu tiên hiệu năng; RAID 1 ưu tiên tính sẵn sàng của dữ liệu.



RAID 5 và RAID 6: phân tán dữ liệu và parity

Parity được phân bố luân phiên để không tạo một đĩa parity cố định.

RAID 5 – một parity

D1	D2	D3	D4
A1	A2	A3	P(A)
B1	B2	P(B)	B3
C1	P(C)	C2	C3

Tối thiểu 3 đĩa • chịu hỏng 1 đĩa • dung lượng sử dụng $(N-1) \times S$

RAID 6 – hai parity

D1	D2	D3	D4
A1	A2	P(A)	Q(A)
B1	P(B)	Q(B)	B2
P(C)	Q(C)	C1	C2

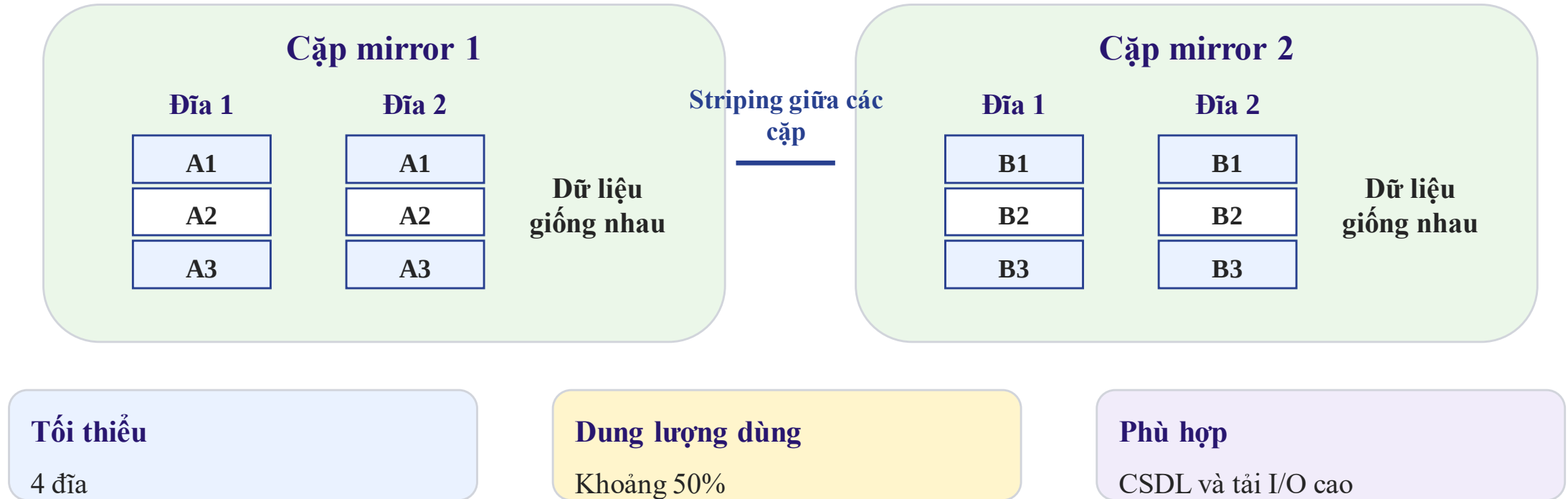
Tối thiểu 4 đĩa • chịu hỏng 2 đĩa • dung lượng sử dụng $(N-2) \times S$

RAID 6 an toàn hơn khi mảng lớn, nhưng ghi parity và phục hồi thường tốn nhiều thời gian hơn.



RAID 10: ghép mirror rồi striping

RAID 10 kết hợp khả năng chịu lỗi của RAID 1 với hiệu năng song song của RAID 0.



So sánh nhanh các mức RAID phổ biến



Mức	Số đĩa tối thiểu	Dung lượng sử dụng	Chịu lỗi	Điểm mạnh	Hạn chế chính
RAID 0	2	$N \times S$	0	Tốc độ cao	Không có dự phòng
RAID 1	2	$\approx 50\%$	1 đĩa/cặp	Đơn giản, khôi phục nhanh	Tốn dung lượng
RAID 5	3	$(N-1) \times S$	1 đĩa	Cân bằng dung lượng và chịu lỗi	Ghi parity, rebuild nặng
RAID 6	4	$(N-2) \times S$	2 đĩa	An toàn hơn cho mảng lớn	Ghi chậm hơn RAID 5
RAID 10	4	$\approx 50\%$	Tùy vị trí hỏng	Nhanh và phục hồi tốt	Chi phí cao

N: số đĩa trong mảng; S: dung lượng của đĩa nhỏ nhất.



RAID không thay thế cho sao lưu

RAID bảo vệ tốt trước

- ✓ Hỏng một hoặc nhiều ổ đĩa
- ✓ Giảm đoạn do thay ổ và phục hồi
- ✓ Nhu cầu duy trì dịch vụ liên tục

RAID không bảo vệ khỏi

- ✗ Xóa nhầm hoặc ghi đè dữ liệu
- ✗ Mã độc tổng tiền và lỗi phần mềm
- ✗ Cháy, ngập, mất cấp hoặc lỗi toàn hệ thống
- ✗ Bản sao hỏng do lỗi logic đã lan truyền

Cần kết hợp RAID với sao lưu định kỳ, bản sao ngoại tuyến và bản sao ở vị trí khác.



HDD và SSD khác nhau trong quản lý vào/ra

HDD – đĩa từ cơ khí

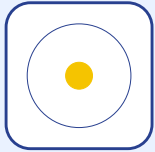
- Thời gian truy cập phụ thuộc vị trí.
- Lập lịch giảm quãng đường đầu đọc.
- Dữ liệu tuần tự thường nhanh hơn ngẫu nhiên.
- Có tiếng ồn, rung và bộ phận chuyển động.
- Giá trên mỗi GB thường thấp.

SSD / NVMe – bộ nhớ flash

- Không có đầu đọc cơ khí.
- Độ trễ ít phụ thuộc vị trí khối.
- Cần TRIM, wear leveling và garbage collection.
- NVMe hỗ trợ nhiều hàng đợi song song.
- Nhanh nhưng có giới hạn chu kỳ ghi.

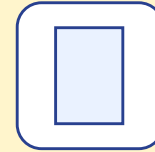
Thuật toán tối ưu cho HDD không nhất thiết mang lại cùng lợi ích trên SSD/NVMe.

Các thiết bị và mô hình lưu trữ khác



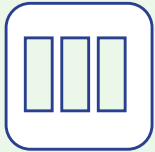
Đĩa quang

CD, DVD, Blu-ray; thuận tiện phát hành hoặc lưu bản sao ít thay đổi.



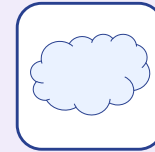
USB / thẻ nhớ

Nhỏ gọn, dễ di chuyển; cần tháo an toàn để tránh mất dữ liệu.



NAS / SAN

Lưu trữ qua mạng; dùng chung cho nhiều máy và dễ mở rộng tập trung.



Lưu trữ đám mây

Truy cập qua dịch vụ mạng; linh hoạt nhưng phụ thuộc băng thông và chính sách nhà cung cấp.

Mỗi loại lưu trữ phù hợp một yêu cầu khác nhau về tốc độ, chi phí, độ bền và khả năng chia sẻ.

Lựa chọn thiết bị lưu trữ theo nhu cầu



Loại	Truy cập ngẫu nhiên	Tốc độ	Chi phí/GB	Khả năng di chuyển	Ứng dụng điển hình
HDD	Tốt	Trung bình	Thấp	Thấp	Kho dữ liệu dung lượng lớn
SSD / NVMe	Rất tốt	Rất cao	Cao hơn HDD	Thấp	Hệ điều hành, CSDL, ứng dụng
Băng từ	Kém	Cao khi đọc tuần tự	Rất thấp	Trung bình	Sao lưu và lưu trữ dài hạn
USB / thẻ nhớ	Tốt	Tùy loại	Trung bình	Rất cao	Trao đổi và mang theo dữ liệu
NAS / Cloud	Phụ thuộc mạng	Phụ thuộc mạng	Theo hệ thống/dịch vụ	Truy cập từ xa	Chia sẻ, đồng bộ, mở rộng

Không có thiết bị nào tốt nhất cho mọi trường hợp; hệ thống thường kết hợp nhiều tầng lưu trữ.

Cấp phát và đặt trước thiết bị



Cấp ngay

Nếu thiết bị rồi và yêu cầu hợp lệ, hệ điều hành cấp quyền sử dụng.

Đưa vào hàng đợi

Nếu thiết bị bận, yêu cầu chờ theo chính sách.

Đặt trước

Một số công việc cần giữ thiết bị trong cả phiên để tránh bị chen ngang.

Giữ thiết bị quá lâu làm giảm khả năng chia sẻ; thu hồi quá sớm có thể làm công việc dang dở.

Thiết bị cũng có thể tham gia bế tắc



Nguyên nhân

Tiến trình giữ một thiết bị và xin thêm thiết bị khác.

Phòng tránh

Yêu cầu theo thứ tự hoặc không giữ thiết bị khi chờ.

Khắc phục

Hủy công việc, thu hồi thiết bị hoặc yêu cầu người dùng can thiệp.

Độc lập thiết bị giúp chương trình dễ chuyển đổi



Chương trình phụ thuộc thiết bị

- Mã nguồn ghi rõ loại máy in hoặc địa chỉ phần cứng.
- Đổi thiết bị phải sửa chương trình.
- Khó chạy trên máy khác.

Chương trình độc lập thiết bị

- Dùng thao tác chung như open, read, write.
- Driver xử lý chi tiết riêng.
- Dễ thay phần cứng mà không sửa ứng dụng.

Độc lập thiết bị là mục tiêu quan trọng của lớp vào/ra trong hệ điều hành.

Tên thiết bị và tên tệp giúp che giấu địa chỉ phần cứng



Tên logic

Ứng dụng dùng tên như /dev/sda, COM3 hoặc printer-office.

Ánh xạ

Hệ điều hành ánh xạ tên logic tới driver và thiết bị thật.

Linh hoạt

Thiết bị có thể đổi cổng hoặc địa chỉ mà ứng dụng ít bị ảnh hưởng.

Tên logic nên thể hiện mục đích sử dụng thay vì buộc người dùng nhớ địa chỉ thanh ghi hay số ngắt.

Bảo vệ quyền truy cập thiết bị



Đọc

Ai được lấy dữ liệu từ thiết bị?

Ghi

Ai được gửi dữ liệu hoặc thay đổi trạng thái?

Điều khiển

Ai được thay cấu hình, đặt lại hoặc tắt thiết bị?

Độc quyền

Yêu cầu nào được giữ thiết bị trong một phiên?

Hệ điều hành kiểm tra quyền trước khi chuyển yêu cầu cho driver.

Quản lý năng lượng của thiết bị



Lợi ích

Giảm điện năng và nhiệt lượng.

Đánh đổi

Đánh thức thiết bị có thể làm tăng độ trễ.

Chính sách

Dựa vào loại thiết bị, pin, tải và yêu cầu người dùng.

Hot-plug: thêm hoặc rút thiết bị khi máy đang chạy



Ví dụ: cắm USB, tai nghe Bluetooth hoặc bộ chuyển đổi mạng.

Khi rút thiết bị, hệ điều hành phải dừng yêu cầu đang chạy và giải phóng tài nguyên an toàn.

Chiến lược khôi phục khi thiết bị lỗi



Retry

Thử lại số lần giới hạn.

Reset

Đặt lại bộ điều khiển hoặc thiết bị.

Fallback

Chuyển sang thiết bị hoặc đường truyền khác.

Fail

Dừng yêu cầu và báo lỗi rõ ràng.

Không nên thử lại vô hạn vì có thể làm hệ thống treo hoặc che giấu lỗi thật.



9.3. Trình điều khiển thiết bị

Driver là phần mềm hiểu thiết bị cụ thể và cung cấp cho hệ điều hành một giao diện sử dụng thống nhất.

Trình điều khiển thiết bị là gì?



Phần mềm trung gian

Nằm giữa hệ điều hành và bộ điều khiển thiết bị.

Hiểu phần cứng

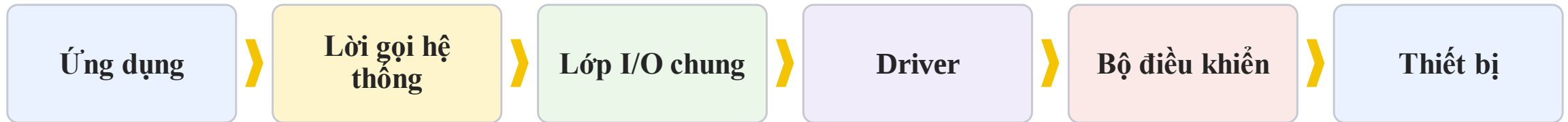
Biết thanh ghi, lệnh, tín hiệu ngắt và cách truyền dữ liệu của thiết bị.

Cung cấp giao diện chung

Biến thao tác read, write, open thành lệnh phần cứng cụ thể.

Không có driver phù hợp, hệ điều hành thường không thể khai thác đầy đủ thiết bị.

Driver nằm ở đâu trong đường đi của yêu cầu?



Lớp I/O chung xử lý phần giống nhau; driver xử lý phần riêng của từng loại thiết bị.

Thiết kế theo lớp giúp thay driver mà ít ảnh hưởng đến ứng dụng.



Các nhiệm vụ chính của driver

Khởi tạo

Phát hiện, kiểm tra và cấu hình thiết bị.

Gửi lệnh

Ghi lệnh và tham số vào bộ điều khiển.

Nhận kết quả

Đọc dữ liệu, trạng thái và mã lỗi.

Xử lý ngắt

Phản hồi khi thiết bị báo hoàn thành hoặc có sự cố.

Quản lý hàng đợi

Sắp xếp các yêu cầu chờ của thiết bị.

Quản lý DMA

Chuẩn bị vùng nhớ và bộ mô tả truyền dữ liệu.

Báo lỗi

Chuyển lỗi phân cứng thành mã lỗi dễ hiểu cho hệ điều hành.



Driver được nạp khi nào?

Nạp sẵn

- Driver nằm trong nhân hoặc được nạp khi khởi động.
- Phù hợp thiết bị thiết yếu như đĩa hệ thống.
- Sẵn sàng sớm nhưng làm nhân lớn hơn.

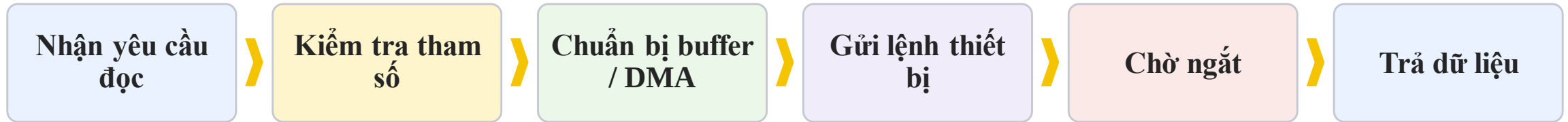
Nạp động

- Driver được nạp khi phát hiện thiết bị hoặc khi cần.
- Tiết kiệm bộ nhớ và dễ cập nhật.
- Cần kiểm tra tính tương thích và chữ ký.

Hệ điều hành hiện đại thường kết hợp cả hai cách.



Ví dụ: driver xử lý một yêu cầu đọc



Nếu thiết bị đang bận, driver đưa yêu cầu vào hàng đợi. Khi hoàn thành, driver cập nhật số byte đã đọc và đánh thức tiến trình đang chờ.



Ví dụ: driver xử lý một yêu cầu ghi



Một lời gọi ghi có thể trả về trước khi dữ liệu thật sự xuống thiết bị nếu hệ thống dùng cache hoặc write-back.



Hàm xử lý ngắt của driver nên làm gì?

Xác nhận ngắt

Đọc trạng thái và báo cho thiết bị biết ngắt đã được nhận.

Lưu kết quả

Cập nhật số byte, mã lỗi và trạng thái yêu cầu.

Đánh thức công việc

Báo tiến trình chờ hoặc lên lịch phân xử lý dài hơn.

Hàm ngắt cần ngắn; công việc dài nên chuyển sang phần xử lý sau.

Driver phối hợp với DMA



Địa chỉ đúng

DMA cần địa chỉ vật lý hoặc địa chỉ I/O phù hợp.

An toàn bộ nhớ

Không được để tiến trình thay đổi vùng đang truyền theo cách gây lỗi.

Đồng bộ cache

Một số hệ thống phải làm sạch hoặc vô hiệu cache trước/sau DMA.

Driver thiết bị khối và thiết bị ký tự



Thiết bị khối

- Dữ liệu theo các khối có kích thước xác định.
- Thường hỗ trợ truy cập ngẫu nhiên.
- Có cache và hàng đợi khối.
- Ví dụ: HDD, SSD.

Thiết bị ký tự

- Dữ liệu theo dòng byte hoặc ký tự.
- Thường xử lý tuần tự.
- Có thể không cần tìm vị trí.
- Ví dụ: bàn phím, cổng nối tiếp.

Card mạng thường có mô hình driver riêng vì làm việc theo gói tin.



Driver có thể chạy trong nhân hoặc ngoài nhân

Driver trong nhân

- Truy cập phần cứng nhanh và trực tiếp.
- Hiệu năng cao.
- Lỗi driver có thể làm toàn hệ thống sập.

Driver ngoài nhân

- Chạy trong tiến trình được cách ly.
- Lỗi ít ảnh hưởng đến nhân.
- Có thêm chi phí giao tiếp giữa các miền.

Lựa chọn phụ thuộc kiến trúc hệ điều hành và yêu cầu hiệu năng, an toàn.



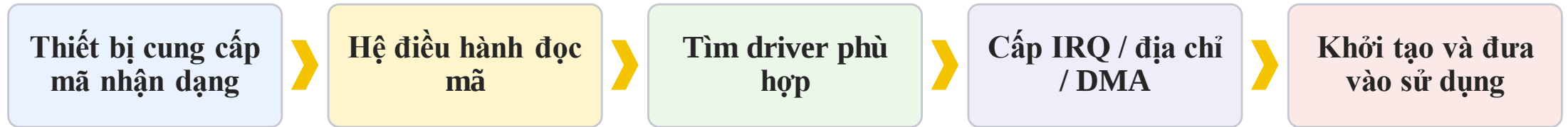
Giao diện driver giúp lớp trên gọi đúng chức năng

```
open(device)    // mở và kiểm tra quyền  
close(device)   // đóng và giải phóng tài nguyên  
read(buffer, n) // đọc dữ liệu  
write(buffer, n) // ghi dữ liệu  
ioctl(command) // lệnh điều khiển riêng  
interrupt()     // xử lý tín hiệu ngắt
```

Tên hàm có thể khác giữa các hệ điều hành, nhưng ý tưởng chung tương tự.



Plug and Play tìm driver phù hợp như thế nào?



Nếu không có driver chính xác, hệ điều hành có thể dùng driver chung với tính năng hạn chế.

Driver là thành phần nhạy cảm về an toàn



Quyền cao

Driver thường có quyền truy cập bộ nhớ và phần cứng.

Dữ liệu đầu vào

Thiết bị có thể gửi dữ liệu sai hoặc bất thường.

Kiểm tra biên

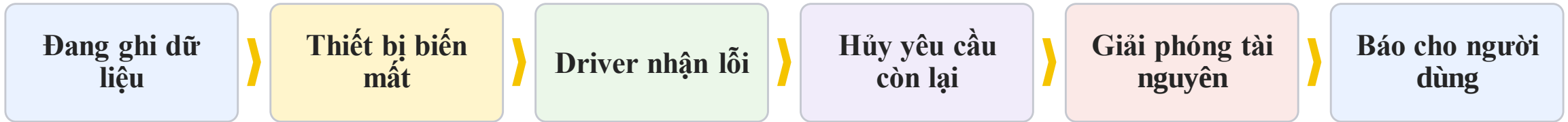
Phải kiểm tra kích thước, địa chỉ và quyền truy cập.

Cập nhật

Cần vá lỗi và xác minh nguồn driver.

Một lỗi nhỏ trong driver có thể gây mất dữ liệu hoặc làm hệ thống không ổn định.

Tình huống: USB bị rút khi đang sao chép dữ liệu



Rủi ro

Tệp có thể chưa ghi đủ hoặc hệ thống tệp chưa cập nhật hoàn chỉnh.

Vai trò driver

Dừng DMA, hủy ngắt, đánh dấu thiết bị ngoại tuyến.

Vai trò hệ điều hành

Báo lỗi, tháo thiết bị logic và bảo vệ cấu trúc dữ liệu.

Ba lớp phối hợp để phục vụ một yêu cầu thiết bị



Lớp	Câu hỏi chính	Ví dụ công việc
Lớp I/O chung	Yêu cầu có hợp lệ và được phép không?	Kiểm tra quyền, chọn thiết bị, quản lý buffer
Driver	Thiết bị cụ thể cần lệnh nào?	Ghi thanh ghi, thiết lập DMA, xử lý ngắt
Bộ điều khiển / thiết bị	Làm thế nào thực hiện công việc vật lý?	Đọc khối, in giấy, truyền gói tin

Tách lớp giúp hệ thống dễ mở rộng và thay đổi thiết bị.

Tổng kết Chương 9



Thiết bị vào/ra hoạt động qua bộ điều khiển, thanh ghi, ngắt, DMA và vùng đệm.

Hệ điều hành dùng bảng thiết bị, hàng đợi và driver để cấp phát, bảo vệ và xử lý lỗi.

Đĩa từ được tổ chức theo rãnh, sector và khối luận lý; băng từ phù hợp truy cập tuần tự và sao lưu.

SCSI cung cấp mô hình lệnh initiator–target–LUN cho các thiết bị lưu trữ.

FCFS, SSTF, SCAN, C-SCAN, LOOK và C-LOOK sắp xếp yêu cầu truy cập HDD.

RAID dùng striping, mirroring và parity để cân bằng hiệu năng, dung lượng và khả năng chịu lỗi.

SSD/NVMe, đĩa quang, USB, NAS và đám mây có đặc điểm quản lý và phạm vi sử dụng khác nhau.



Câu hỏi thảo luận

Tại sao hệ điều hành không cho ứng dụng điều khiển trực tiếp thanh ghi thiết bị?

Trong trường hợp nào FCFS có thể tốt hơn một thuật toán tối ưu quãng đường?

Vì sao driver thường là một trong những nguyên nhân làm hệ điều hành mất ổn định?

Thiết bị ảo giúp việc chia sẻ và kiểm thử thuận tiện như thế nào?

Nếu ưu tiên tuyệt đối cho tác vụ thời gian thực, các yêu cầu thông thường có thể gặp vấn đề gì?

Email: minhnl@utc.edu.vn